SIMATS ENGINEERING

Department of Information Technology

DSA0116 – Object-Oriented Programming with C++

ASSIGNMENT 2 – CO2

Design C++ Programs by Applying Functions and Object-Oriented Concepts

CO2: Design C++ programs by applying functions and object-oriented concepts — function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Field	Details	Field	Details
Student Name	Rekha Mondal	**Register Number**	192521292
Department / Section	Btech IT	**Academic Year / Semester**	2025
Faculty Name		**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	12/11/2026	**Date of Evaluation**	
GitHub Repository	https://github.com/Rekha28wp/DSA0116_192521292	**Compiler**	g++ 13.3.0, -std=c++17 -Wall -Wextra

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
Q1	Call by Value, by Reference and by Pointer	6		
Q2	Return by Reference and Its Dangers	4		
Q3	Default Arguments and Function Overloading	6		
Q4	Specifying a Class with an Array Inside It	6		
Q5	Static Members and Arrays of Objects	7		
Q6	Friend Functions and Inline Functions	8		
Q7A	Diagnose the Limitation	2		

Q7B	Define the Design Objective	2		
Q7C	Design the Revised Solution	4		
Q7D	Predict Before Implementing	1		
Q7E	Implement and Validate the Redesign	2		
Q7F	Defend the Final Design	2		
	TOTAL	**50**		**Percentage:**

OVERALL REMARKS:

OBJECT-ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 2 – CO2

- **Nature of Assignment:** Individual
- **Duration:** 1 Week
- **Total Marks:** 50
- **Submission Date:** 12/11/2026
- **Bloom's Levels Assessed:** K5 – Evaluate (every task in this assignment is at K5 level)
- **GitHub Repository Link:** https://github.com/Rekha28wp/DSA0116_192521292

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

Bloom's Level: K5 – Evaluate

Problem Context: Hostel Fee Concession Updater

Ans:

1. Three Implementations of the Concession Function

To eliminate floating-point rounding inaccuracies, all hostel fees are represented in integer arithmetic (long long). Below are the three parameter-passing mechanisms for applying merit (10%), sports (15%), and single-parent (25%) concessions:

C++
```
#include <iostream>

// 1. Call by Value: receives an independent copy of the caller's argument
void applyByValue(long long fee, int percent) {
```

```
    fee -= fee * percent / 100;
    std::cout << "  [Inside applyByValue]     fee = " << fee << '\n';
}

// 2. Call by Reference: 'fee' becomes an alias for the caller's variable
void applyByReference(long long& fee, int percent) {
    fee -= fee * percent / 100;
    std::cout << "  [Inside applyByReference] fee = " << fee << '\n';
}

// 3. Call by Pointer: 'feePtr' stores the memory address of the caller's variable
void applyByPointer(long long* feePtr, int percent) {
    if (feePtr != nullptr) {
        *feePtr -= (*feePtr) * percent / 100;
        std::cout << "  [Inside applyByPointer]   *feePtr = " << *feePtr << '\n';
    }
}

int main() {
    long long fee1 = 45000, fee2 = 45000, fee3 = 45000;

    // Merit concession: 10%
    applyByValue(fee1, 10);
    std::cout << "Stored fee1 after call by value:     " << fee1 << "\n\n";

    // Sports concession: 15%
    applyByReference(fee2, 15);
    std::cout << "Stored fee2 after call by reference: " << fee2 << "\n\n";

    // Single parent concession: 25%
    applyByPointer(&fee3, 25);
    std::cout << "Stored fee3 after call by pointer:   " << fee3 << '\n';
    return 0;
}
```

2. Predictions Before Running vs. Observed Execution Results

- **Predictions (written prior to compilation):**
    - applyByValue(fee1, 10): Inside the function, fee becomes 45000 - 4500 = 40500, but fee1 in main() remains **45000**.
    - applyByReference(fee2, 15): Both inside and in main(), fee2 becomes 45000 - 6750 = **38250**.
    - applyByPointer(&fee3, 25): Both inside and in main(), fee3 becomes 45000 - 11250 = **33750**.
- **Observed Output (g++ -std=c++17 -Wall -Wextra q1_three_ways.cpp -o q1_three_ways):**

Plaintext
[Inside applyByValue] fee = 40500
Stored fee1 after call by value: 45000

[Inside applyByReference] fee = 38250
Stored fee2 after call by reference: 38250

[Inside applyByPointer] *feePtr = 33750
Stored fee3 after call by pointer: 33750

3. Inquiry: Storage Location, Caller Mutation, and const References

Mechanism	Where the Parameter is Stored	Why Caller's Variable Changes (or Does Not)
Call by Value (long long fee)	Allocated as a brand-new local variable inside the callee's stack frame, initialized with a bit-for-bit copy of the caller's value.	Arithmetic modifies only the callee's private stack copy. When the function returns, its stack frame is popped and destroyed, leaving the caller's memory untouched.
Call by Reference (long long& fee)	Bound directly at compile time as an alias to the caller's original memory address (typically implemented under the hood as an implicit hidden pointer).	Any read or write to fee accesses the caller's original stack/heap memory location directly, mutating the caller's state in place.
Call by Pointer (long long* feePtr)	feePtr itself is a local 8-byte variable on the callee's stack frame holding the *memory address* (&fee3) of the caller's variable.	Dereferencing (*feePtr) follows that address back to the caller's memory location and modifies the value stored there.

- **When to use a const reference (const T&):** A const reference should be used whenever a function needs **read-only access** to an object—especially large user-defined objects like Student or std::string—where copying would waste memory and CPU cycles, yet the function must be strictly prohibited by the compiler from modifying the caller's data (for example, void printReceipt(const Student& s)).

4. Design Decision: In-Place Mutation vs. Returning a New Fee

- **Decision:** The concession function should **return the new fee** (long long computeConcession(long long fee, int percent)) rather than mutating the fee in place.
- **Defense:**
  - **Previewing and Auditing:** When multiple concessions apply (e.g., Merit 10% + Sports 15%), the hostel office often needs to preview different concession combinations, verify whether sequential compounding vs. additive percentage rules apply, or reject an invalid concession without corrupting the stored balance mid-transaction.
  - **Referential Transparency & Safety:** A pure function taking fee and percent and returning the discounted value has no side effects, prevents accidental double-deductions if called twice, and

works seamlessly with const variables or temporary expressions (long long finalFee = computeConcession(computeConcession(baseFee, 10), 15);).

5. the Output (Given Program)

- **Output (g++ -std=c++17 -Wall -Wextra q1_predict.cpp -o q1_predict):**

Plaintext
1 12 13
1 100
16
13
6

- **Detailed Explanation of Every Call and Default Argument:**
  - change(a, b, &c):
    - a is passed **by value** (int a), so a += 10 modifies only the local copy; main's a stays **1**.
    - b is passed **by reference** (int& b), so b += 10 directly mutates main's b from 2 to **12**.
    - c is passed **by pointer** (int* c), so *c += 10 dereferences &c and mutates main's c from 3 to **13**.
  - bigger(a, c) = 100:
    - bigger takes p (a = 1) and q (c = 13) by reference. Since 1 > 13 is false, bigger returns an lvalue reference (int&) bound directly to c.
    - Assigning = 100 to that returned reference overwrites c in main() with **100**, while a remains **1**.
  - show(1): x = 1 is supplied; compiler fills both trailing defaults y = 5 and z = 10 $\rightarrow 1+5+10=\backslash rightarrow\ 1\ +\ 5\ +\ 10\ =\rightarrow 1+5+10=$ **16**.
  - show(1, 2): x = 1 and y = 2 are supplied; compiler fills only the rightmost default z = 10 $\rightarrow 1+2+10=\backslash rightarrow\ 1\ +\ 2\ +\ 10\ =\rightarrow 1+2+10=$ **13**.
  - show(1, 2, 3): All three arguments x = 1, y = 2, and z = 3 are explicitly supplied; no default arguments are used $\rightarrow 1+2+3=\backslash rightarrow\ 1\ +\ 2\ +\ 3\ =\rightarrow 1+2+3=$ **6**.

6. Complete the Code

C++
```
#include <iostream>

void applyByValue(long long fee, int percent)
{
   fee -= fee * percent / 100; // changes only the local copy
}

void applyConcession(long long& fee, int percent)
{
```

```
    // TODO 1: Reduce the caller's fee by percent using integer arithmetic (call by reference)
    fee -= fee * percent / 100;
}

int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value: " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

- **Observed Output:**

Plaintext
after call by value: 45000
after call by reference: 40500

- **Why call by value leaves the fee at 45000:** In applyByValue, the parameter long long fee is allocated as an independent 8-byte variable in applyByValue's stack frame containing a copy of 45000. The expression fee -= fee * percent / 100 updates that local copy to 40500, which is discarded when the function returns. hostelFee in main() is never accessed.

7. Alternative Concept and Complexity Analysis

- **Alternative Solution (Return by Value):**

C++
```
long long applyConcession(long long fee, int percent) {
    return fee - (fee * percent / 100);
}
// Caller usage: hostelFee = applyConcession(hostelFee, 10);
```

- **Complexity & Memory Copy Comparison (long long vs. Large Student Object):**

Parameter Type	Time Complexity	Space Copied per Call (long long, 8 B)	Space Copied per Call (Student struct, e.g., 256 B)	Mutation Safety
By Value (T obj)	O(1) per word copied (O(sizeof(T)))	8 bytes (1 CPU register)	**256 bytes** + deep copies of internal std::strings	Safe (caller isolated), but expensive for large objects

By Reference (T& obj)	O(1)	8 bytes (memory address)	**8 bytes** (memory address only)	Can mutate caller's object
By const Reference (const T& obj)	O(1)	8 bytes (memory address)	**8 bytes** (memory address only)	**Zero copy + compiler-enforced read-only safety**

Q2 – Return by Reference and Its Dangers (4 Marks)

Bloom's Level: K5 – Evaluate

Problem Context: Examination Marks Register

Ans:

1. Implementation Demonstrating Safe Array Reference vs. Dangling Local Reference

C++

```
#include <iostream>

// Safe: returns a reference to an array element whose lifetime outlives the function
int& highestMark(int marks[], int n) {
    int maxIdx = 0;
    for (int i = 1; i < n; ++i) {
        if (marks[i] > marks[maxIdx]) {
            maxIdx = i;
        }
    }
    return marks[maxIdx];
}

// Dangerous: returns a reference to a local automatic variable destroyed on return
int& badLocalRef(int x) {
    int temp = x + 10;
    return temp; // Compiler warns: reference to local variable 'temp' returned
}

int main() {
    int marks[5] = {72, 88, 91, 65, 84};

    // 1. Modifying the array element through the returned reference (+2 moderation)
    highestMark(marks, 5) += 2;
    std::cout << "After +2 moderation, marks[2] = " << marks[2] << '\n';
```

```
    // 2. Using the function call directly on the left-hand side of '='
    highestMark(marks, 5) = 100;
    std::cout << "After capping at 100, marks[2] = " << marks[2] << '\n';

    return 0;
}
```

2. How Returning a Reference Works & Dangling Reference Undefined Behaviour

- **Left-hand side of =:** Because highestMark returns an lvalue reference (int&) bound directly to marks[2] in the caller's array, the expression highestMark(marks, 5) evaluates to the memory location of marks[2] itself rather than a temporary rvalue. Thus, highestMark(marks, 5) += 2 and highestMark(marks, 5) = 100 write directly into marks[2].
- **Compiler Warning on Local Variable Reference (g++ -std=c++17 -Wall -Wextra):**

```
Plaintext
q2_marks.cpp: In function 'int& badLocalRef(int)':
q2_marks.cpp:16:12: warning: reference to local variable 'temp' returned [-Wreturn-local-addr]
   16 |     return temp;
      |            ^~~~
```

- **Why using badLocalRef is Undefined Behaviour (UB):** temp has automatic storage duration inside badLocalRef's stack frame. As soon as badLocalRef executes return, its stack frame is popped and reclaimed. The caller receives a **dangling reference** pointing to invalidated stack memory. Reading from it may yield garbage once another function call overwrites that stack region, and writing through it corrupts another function's stack frame or return address.

3. Comparison: int& vs. const int& vs. int (Value)

Return Type	What is Returned	Can Caller Modify Original?	Lifetime / Safety Requirement
int& (Reference)	Lvalue alias to the original memory slot	**Yes** (fn() = 100; is valid)	Referent must outlive the function call
const int& (const Reference)	Read-only lvalue alias to the original slot	**No** (assignment is a compile error)	Referent must outlive the function call
int (Copy / Value)	A brand-new temporary rvalue copy	**No** (fn() = 100; is a compile error)	Always safe, even for local variables

4. Changed Condition: std::vector Reallocation Invalidation

If the marks register is stored in a std::vector<int> marks and a caller saves int& top = highestMark(marks); before calling marks.push_back(95);:

- When marks.size() == marks.capacity(), push_back() allocates a **new, larger contiguous heap buffer**, moves/copies all existing elements into the new buffer, and **frees the old heap memory**.
- The previously captured reference top still points to the **deallocated old heap address**. Any subsequent read or write through top triggers heap use-after-free undefined behaviour.

5. Alternative Concept and Complexity Analysis (int Index vs. int Pointer vs. int& Reference)*

- **Calling Code Comparison:**

C++
```
// 1. Returning an index (int)
int idx = highestMarkIndex(marks);
marks[idx] += 2; // Remains valid even after marks.push_back() reallocates!

// 2. Returning a pointer (int*)
int* ptr = highestMarkPtr(marks);
if (ptr != nullptr) *ptr += 2; // Invalidated on vector reallocation (dangling pointer)
```

- **Complexity and Safety Analysis:**

Approach	Time to Find Max	Auxiliary Space	Safety When std::vector Grows
Reference (int&)	O(n)	O(1)	**Unsafe:** Dangles upon heap reallocation; cannot represent an empty register (n == 0) without throwing.
Pointer (int*)	O(n)	O(1)	**Unsafe:** Dangles upon heap reallocation, though it can safely return nullptr for an empty register.
Index (int / size_t)	O(n)	O(1)	**Safe:** An integer offset (marks[idx]) is decoupled from raw memory addresses and remains valid across vector reallocations.

Q3 – Default Arguments and Function Overloading (6 Marks)

Bloom's Level: K5 – Evaluate

Problem Context: Courier Charge Calculator

Ans:

1. Explanation of the Ambiguous Call Compiler Error

In the original code, the developer declared:

1. double charge(double weight);
2. double charge(double weight, bool express);
3. double charge(double weight, bool express = false, bool insured = false);

Because the third overload provides default values for **both** express and insured, it acts as three callable signatures simultaneously:

- charge(double) — collides identically with Overload #1!
- charge(double, bool) — collides identically with Overload #2!
- charge(double, bool, bool)

When the caller invokes charge(2.5) or charge(2.5, true), both Overload #1/#2 and Overload #3 are **exact matches** with identical conversion ranks. The C++ overload resolution engine cannot prefer one over the other and aborts compilation with error: call of overloaded 'charge(double)' is ambiguous.

2. Redesigned Overloaded charge Functions (Zero Ambiguity)

By removing the redundant 1-parameter and 2-parameter double overloads and distinguishing input types cleanly (int grams, double kilograms, and const Parcel& struct), every call resolves uniquely:

C++

```
#include <iostream>

struct Parcel {
    double weightKg;
    bool express;
    bool insured;
};

// Overload 1: Weight in kilograms (double) with trailing default arguments
double charge(double weightKg, bool express = false, bool insured = false) {
    double cost = weightKg * 50.0; // Rs 50 per kg
    if (express) cost += 40.0;
    if (insured) cost += 25.0;
    return cost;
}

// Overload 2: Weight in grams (int) with trailing default arguments
double charge(int weightGrams, bool express = false, bool insured = false) {
    return charge(weightGrams / 1000.0, express, insured);
}

// Overload 3: Complete Parcel object (const reference to avoid copying)
double charge(const Parcel& p) {
    return charge(p.weightKg, p.express, p.insured);
}

int main() {
    Parcel box{3.5, true, false};
```

```
    std::cout << "1. charge(2.5):             " << charge(2.5) << '\n';
    std::cout << "2. charge(2.5, true):       " << charge(2.5, true) << '\n';
    std::cout << "3. charge(2.5, true, true): " << charge(2.5, true, true) << '\n';
    std::cout << "4. charge(2):               " << charge(2) << '\n';
    std::cout << "5. charge(2.0f):            " << charge(2.0f) << '\n';
    std::cout << "6. charge(box):             " << charge(box) << '\n';
    return 0;
}
```

3. Six Designed Calls: Pre-Execution Predictions vs. Observed Output

SI.NO	Call Expression	Predicted Overload Selected & Match Rule	Predicted Output	Actual Output
1	charge(2.5)	charge(double, bool=false, bool=false) — **Exact Match** (2.5 is double)	125	125
2	charge(2.5, true)	charge(double, bool, bool=false) — **Exact Match** (double, bool)	165	165
3	charge(2.5, true, true)	charge(double, bool, bool) — **Exact Match** (all 3 arguments explicit)	190	190
4	charge(2)	charge(int, bool=false, bool=false) — **Exact Match** (2 is int, treated as 2 grams = 0.002 kg)	0.1	0.1
5	charge(2.0f)	charge(double, bool=false, bool=false) — **Floating-Point Promotion** (float → double outranks standard conversion float → int)	100	100
6	charge(box)	charge(const Parcel&) — **Exact Match** (user-defined type Parcel)	215	215

4. Inquiry: Overload Resolution Hierarchy & Right-to-Left Default Arguments

- **How the Compiler Ranks Candidate Overloads (from highest priority to lowest):**
  - **Exact Match:** Argument type matches parameter type directly (including lvalue-to-rvalue conversion or adding top-level const).
  - **Promotion:** Widening conversions that preserve value without loss of precision (char/short →*rightarrow*→ int, float →*rightarrow*→ double). Thus charge(2.0f) promotes float to double rather than converting to int.
  - **Standard Conversion:** Lossy or cross-category built-in conversions (int →*rightarrow*→ double, double →*rightarrow*→ int, int →*rightarrow*→ bool).
  - **User-Defined Conversion:** Single-argument non-explicit constructors or conversion operators.
  - **Ellipsis (...):** Lowest-priority C-style variadic match.
- **Why default arguments must be given from the right:** C++ matches arguments positionally from left to right. If a middle parameter had a default value while a parameter to its right did not (e.g., void f(int a, int b = 5, int c)), a call like f(1, 2) would be syntactically impossible to disambiguate—the compiler

could not tell whether 2 is meant for b (with c missing) or for c (with b defaulted). Enforcing rightmost-first defaults guarantees a contiguous prefix of supplied arguments.

5. Alternative Concept and Complexity Analysis

- **Alternative Solution (Single Configuration Struct with C++11 Default Member Initializers):**

C++
```
struct CourierOptions {
   double weightKg = 0.0;
   bool express = false;
   bool insured = false;
   bool fragile = false; // Adding a new option requires ZERO new overloads!
};

double calculateCharge(const CourierOptions& opt) {
   double cost = opt.weightKg * 50.0;
   if (opt.express) cost += 40.0;
   if (opt.insured) cost += 25.0;
   if (opt.fragile) cost += 30.0;
   return cost;
}
// Caller with C++20 designated initializers: calculateCharge({.weightKg = 2.5, .insured = true});
```

- **Complexity and Extensibility Comparison:**
  - **Time Complexity:** Overload resolution occurs **100% at compile time** (*O(1)O(1)*O(1) runtime cost in both designs; zero runtime dispatch overhead).
  - **Space Copied per Call:** Overloaded primitive calls copy 8–10 bytes in CPU registers; passing const CourierOptions& copies a single 8-byte pointer regardless of how many option fields exist in the struct.
  - **Adding a New Option (e.g., fragile or giftWrap):** With boolean default arguments, adding *kk*k boolean options creates combinatorial explosion (*2k2^k*2k combinations) and forces callers to pass dummy false arguments just to reach the rightmost parameter (e.g., charge(2.5, false, false, true)). With a single CourierOptions struct, adding a new option adds one defaulted field without breaking any existing call site.

Q4 – Specifying a Class with an Array Inside It (6 Marks)

Bloom's Level: K5 – Evaluate

Problem Context: Savings Account Mini-Statement

Ans:

1. Complete Class Specification (SavingsAccount)

Below is the complete implementation demonstrating member functions defined both **inside** the class body (implicitly inline) and **outside** using the scope resolution operator (SavingsAccount::):

C++

```
#include <iomanip>
#include <iostream>
#include <string>

struct Transaction {
   std::string type; // "DEPOSIT" or "WITHDRAW"
   long long amount; // stored in rupees/paise
};

class SavingsAccount {
private:
   static constexpr int MAX_HISTORY = 10;
   long long accountNumber_;
   long long balance_;
   Transaction history_[MAX_HISTORY]; // fixed-size array inside class
   int count_;                        // number of stored transactions (0..10)

   // Private helper defined inside the class
   void recordTransaction(const std::string& type, long long amount) {
      if (count_ < MAX_HISTORY) {
         history_[count_++] = {type, amount};
      } else {
         // Shift elements left by 1 to discard the oldest transaction at index 0
         for (int i = 1; i < MAX_HISTORY; ++i) {
            history_[i - 1] = history_[i];
         }
         history_[MAX_HISTORY - 1] = {type, amount};
      }
   }
```

```
public:
    // Constructor defined inside the class
    SavingsAccount(long long accNo, long long initialBalance)
        : accountNumber_(accNo), balance_(initialBalance), count_(0) {}

    // Inline getter defined inside the class
    long long balance() const { return balance_; }

    // Member functions declared inside and defined outside using '::'
    bool deposit(long long amount);
    bool withdraw(long long amount);
    void miniStatement() const;
};

// External member function definitions using scope resolution operator (::)
bool SavingsAccount::deposit(long long amount) {
    if (amount <= 0) return false;
    balance_ += amount;
    recordTransaction("DEPOSIT", amount);
    return true;
}

bool SavingsAccount::withdraw(long long amount) {
    if (amount <= 0 || amount > balance_) return false;
    balance_ -= amount;
    recordTransaction("WITHDRAW", amount);
    return true;
}

void SavingsAccount::miniStatement() const {
    std::cout << "=== Mini-Statement (Acc: " << accountNumber_
              << " | Balance: Rs " << balance_ << ") ===\n";
    for (int i = 0; i < count_; ++i) {
        std::cout << std::setw(2) << (i + 1) << ". "
                  << std::left << std::setw(10) << history_[i].type
                  << " Rs " << history_[i].amount << '\n';
    }
}

int main() {
    SavingsAccount acc(10045892, 5000);
    for (int i = 1; i <= 12; ++i) {
        acc.deposit(i * 100);
    }
```

```
    acc.withdraw(1500);
    acc.miniStatement(); // Displays transactions 4..13 (last 10 only)
    return 0;
}
```

2. Inquiry: Why the Array Must Be private & Handling the 11th Transaction

- **Why history_ and count_ must be private:**
  - **Class Invariant Protection:** The class maintains strict invariants: 0 <= count_ <= MAX_HISTORY, balance_ equals the initial deposit plus the net sum of all recorded transactions, and no out-of-bounds index is ever accessed. If history_ or count_ were public, external code could set count_ = 99 (causing buffer-overflow undefined behaviour in miniStatement()) or inject fake deposit entries without updating balance_.
  - **Representation Independence:** Keeping history_ private allows switching from a shifting array to a circular buffer internally without breaking any ATM client code calling deposit(), withdraw(), or miniStatement().
- **11th Transaction Handling (Shifting Array vs. Circular Index):**
  - **Shifting Array:** When count_ == 10, indices 1..9 are copied one slot left (history_[i - 1] = history_[i]), overwriting index 0 (oldest), and the new transaction is placed at index 9. This costs ***O(k)O(k)*****O(k)** assignments (*k=10k = 10*k=10 elements shifted per transaction).
  - **Circular Index (Ring Buffer):** Maintain a start_ index (0..9). On the 11th transaction, overwrite history_[start_] directly and advance start_ = (start_ + 1) % MAX_HISTORY. No elements are moved, costing ***O(1)O(1)*****O(1)** time.

3. Design Decision: Scaling from 10 to 50 Transactions

- **Decision:** The **Circular Buffer design** (parameterized by static constexpr int MAX_HISTORY = 50;) changes the least and is superior.
- **Defense:**
  - **Minimal Code Change:** Changing MAX_HISTORY from 10 to 50 requires editing **one single constant** in both designs.
  - **Runtime Performance Scaling:** Under the shifting design, every transaction after the 50th performs *494*49 Transaction struct copies (including heap-allocated std::string move/copy operations), making insertion 5× slower. Under the circular buffer design, insertion remains strictly ***O(1)O(1)*****O(1)** (1 overwrite + 1 modulo increment) regardless of whether MAX_HISTORY is 10, 50, or 10,000.

4. Alternative Concept and Complexity Analysis

- **Alternative Solution 1: Circular Buffer Inside Fixed Array (*O(1)O(1)*O(1)):**

C++

```
void recordCircular(const Transaction& tx) {
    if (count_ < MAX_HISTORY) {
```

```
        history_[(start_ + count_) % MAX_HISTORY] = tx;
        ++count_;
    } else {
        history_[start_] = tx;                // overwrite oldest in O(1)
        start_ = (start_ + 1) % MAX_HISTORY;    // advance oldest pointer
    }
}
```

- **Alternative Solution 2: std::deque<Transaction> (*O(1)O(1)*O(1) Amortized):**

```
C++
void recordDeque(const Transaction& tx) {
    if (historyDeque_.size() == MAX_HISTORY) historyDeque_.pop_front();
    historyDeque_.push_back(tx);
}
```

- **Time & Space Complexity Comparison (*k=k* =k= MAX_HISTORY):**

Design	Time to Add Transaction	Time to Print Statement	Space Used	Memory Locality & Overhead
Shifting Fixed Array	O(k)	O(k)	O(k) (k×sizeof(Transaction))	Contiguous stack/inline memory; zero heap allocation
Circular Fixed Array	**O(1)**	O(k)	O(k) (+4 bytes for start_)	**Best:** Contiguous memory, zero heap allocation, O(1) insert
std::deque	O(1) amortized	O(k)	O(k) + map/chunk pointers	Dynamic heap allocations; higher memory overhead

Q5 – Static Members and Arrays of Objects (7 Marks)

Bloom's Level: K5 – Evaluate

Problem Context: College Bus Seat Tickets

Ans:

1. Class Design Principles for Static Members & Arrays of Objects

- **Which members must be static and where they must be defined:**
    - static int issued; and static const int CAPACITY = 3; belong to the class as a whole rather than individual tickets, ensuring a single shared seat counter across all BusTicket instances.

    - static int count(); must be a static member function so the transport office can query BusTicket::count() directly via the class scope without instantiating a dummy ticket object.
    - **Definition location:** In C++17, a non-inline static data member (static int issued;) is only *declared* inside the class definition and **must be defined once at namespace/global scope** (int BusTicket::issued = 0;) in a .cpp file so the linker allocates storage for it.
- **Why a static member function cannot use this or non-static members:**
    - Non-static member functions receive an implicit hidden parameter BusTicket* const this pointing to the specific object on which the method was invoked.
    - A static member function can be called without any object (BusTicket::count()), so no this pointer is passed to it. Without this, the function has no object address from which to read instance-specific fields like seat or name.
- **How an array of BusTicket objects is created and initialised:**
    - If a class has no default constructor, an array of objects is initialized using brace-enclosed aggregate initialization where each element explicitly invokes the parameterized constructor in index order (trip[0], trip[1], trip[2], trip[3]).
- **What happens when tickets are created inside a loop or function:**
    - Because issued has **static storage duration** (created before main() starts and destroyed only when the program terminates), it is **not reset** when local BusTicket objects go out of scope at the end of a loop iteration or function call. Subsequent calls continue incrementing issued until CAPACITY is reached.

2. Predict the Output (Token Program)

- **My Predicted Output (written before running):**

Plaintext
101 103 104 4

- **Actual Output (g++ -std=c++17 -Wall -Wextra q5_predict.cpp -o q5_predict):**

Plaintext
101 103 104 4

- **Explanation:**
    - int Token::next = 100; initializes the single shared static counter to 100 before main() begins.
    - Token counter[3]; invokes the default constructor Token() **3 times** sequentially: counter[0].id = 101, counter[1].id = 102, counter[2].id = 103 (next becomes 103).
    - Token extra; invokes Token() a **4th time**: extra.id = 104 (next becomes 104).
    - Token::issued() returns next - 100 *=104−100=4= 104 - 100 = 4*=104−100=4. Thus the constructor ran **4 times** in total.

3. Complete the Code & Testing Across Two Function Calls

C++

```
#include <iostream>
#include <string>

class BusTicket {
   static int issued; // shared by all BusTicket objects
   static const int CAPACITY = 3;
   int seat;
   std::string name;

public:
   explicit BusTicket(const std::string& n) : seat(0), name(n)
   {
      // TODO 1: If fewer than CAPACITY tickets have been issued, increase issued and
      // give this ticket the next seat number; otherwise leave seat as 0 (waiting list)
      if (issued < CAPACITY) {
         seat = ++issued;
      }
   }

   static int count()
   {
      // TODO 2: Return the number of seats issued so far
      return issued;
   }

   void print() const
   {
      std::cout << name << ": "
                << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
                << '\n';
   }
};

int BusTicket::issued = 0; // definition of the static data member

void issueBatch(const std::string& batchName, const std::string& s1, const std::string& s2) {
   std::cout << "--- " << batchName << " ---\n";
   BusTicket batch[2] = {BusTicket(s1), BusTicket(s2)};
   for (const BusTicket& t : batch) t.print();
```

```
    std::cout << "Issued so far: " << BusTicket::count() << "\n\n";
}

int main()
{
    // Part A: Original expected output verification (array larger than CAPACITY = 3)
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    std::cout << "Issued: " << BusTicket::count() << "\n\n";

    // Part B: Calling a function twice after capacity is already full
    issueBatch("Morning Batch", "Esha", "Farhan");
    issueBatch("Afternoon Batch", "Gita", "Hari");
    return 0;
}
```

- **Observed Output:**

```
Plaintext
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3

--- Morning Batch ---
Esha: Waiting
Farhan: Waiting
Issued so far: 3

--- Afternoon Batch ---
Gita: Waiting
Hari: Waiting
Issued so far: 3
```

- **How the count behaves when tickets are created inside a function called twice:** Even though batch inside issueBatch is a local array destroyed at the end of the first call, BusTicket::issued persists across function invocations. Because issued had already reached CAPACITY (3), every ticket created in both the first and second calls to issueBatch goes straight to Waiting, and BusTicket::count() remains 3.

4. Alternative Concept and Complexity Analysis

- **Alternative 1 (Global Variable):** int g_issued = 0; referenced inside BusTicket. Any unrelated function in the program can overwrite g_issued = -5;, destroying encapsulation, and you still cannot run two separate buses independently.
- **Alternative 2 (Separate TicketOffice Object Passed by Reference):**

```
C++
class TicketOffice {
   int issued_ = 0;
   const int capacity_;
public:
   explicit TicketOffice(int cap) : capacity_(cap) {}
   int allocateSeat() { return (issued_ < capacity_) ? ++issued_ : 0; }
   int count() const { return issued_; }
};
// BusTicket constructor takes (const std::string& n, TicketOffice& office)
```

- **Comparison Matrix:**

Design	Time Complexity	Space Complexity	Encapsulation	Running Two Buses Simultaneously
static Data Member	O(1)	O(1) (1 int for entire class)	**High** (private to BusTicket)	**No:** All BusTicket objects share one global class counter
Global Variable	O(1)	O(1) (1 global int)	**None** (any file can corrupt it)	**No:** Single global state
TicketOffice Object	O(1)	O(1) per bus (1 int per TicketOffice)	**High** (private to TicketOffice)	**Yes:** TicketOffice bus1(40), bus2(50); operate independently

Q6 – Friend Functions and Inline Functions (8 Marks)

Bloom's Level: K5 – Evaluate

Problem Context: Train Journey Planner

Ans:

1. Design Analysis: Friend Functions, inline, and Encapsulation

- **Why a friend function is needed for symmetric binary operations:**
  - Operations like minutesBetween(a, b) and earlier(a, b) compare two Time objects on equal footing. Writing them as non-member functions allows symmetric syntax (minutesBetween(dep, arr)) and applies implicit conversions equally to both operands if ever needed, while granting privileged access to private h and m without exposing setters/getters to the public API.

- **How a friend function differs from a member function:**
  - **Call Syntax:** A member function is invoked on a receiver object via dot notation (a.isEarlier(b)), whereas a friend function is called like a normal non-member function (earlier(a, b)).
  - **this Pointer:** A friend function is **not** in the class scope and has **no this pointer**; it receives objects explicitly via parameters (const Time& a, const Time& b).
  - **Access Rights:** Both member functions and friend functions can access private and protected members of Time, but friend access is explicitly granted by the class declaration (friend ...;).
- **What inline actually asks the compiler to do:**
  - inline is a **non-binding optimization hint** asking the compiler to substitute the function's machine instructions directly at each call site (eliminating CALL/RET stack frame setup and argument pushing overhead), and a **linkage directive** permitting identical definitions to appear across multiple translation units (ODR exemption).
- **What would be lost if h and m were made public instead:**
  - The class would lose its **time invariant** (0 <= h <= 23 and 0 <= m <= 59). Any caller could write t.h = 99; t.m = -45;, causing minutesBetween to produce corrupt travel durations.

2. Complete the Code & Correctness Checks

C++

```
#include <cmath>
#include <cstdlib>
#include <iostream>

class Time {
   int h, m;
public:
   Time(int hh, int mm) : h(hh), m(mm) {}
   friend inline int minutesBetween(const Time& a, const Time& b);
   friend inline bool earlier(const Time& a, const Time& b);
};

inline int minutesBetween(const Time& a, const Time& b)
{
   // TODO 1: Using the private members h and m, return the absolute difference between
   // the two times in minutes
   int totalA = a.h * 60 + a.m;
   int totalB = b.h * 60 + b.m;
   return std::abs(totalA - totalB);
}

inline bool earlier(const Time& a, const Time& b)
{
   // TODO 2: Return true if a is earlier in the day than b
   return (a.h * 60 + a.m) < (b.h * 60 + b.m);
```

```
}

int main()
{
   Time departure(9, 45), arrival(13, 10);
   std::cout << minutesBetween(departure, arrival) << ' '
         << std::boolalpha << earlier(arrival, departure) << '\n';

   // Additional Correctness Checks:
   // 1. Same time twice
   Time t1(12, 0), t2(12, 0);
   std::cout << "Same time (12:00, 12:00): diff = " << minutesBetween(t1, t2)
         << ", earlier = " << earlier(t1, t2) << '\n';

   // 2. Pair of times on either side of noon (11:30 AM and 12:15 PM)
   Time preNoon(11, 30), postNoon(12, 15);
   std::cout << "Across noon (11:30, 12:15): diff = " << minutesBetween(preNoon, postNoon)
         << ", earlier = " << earlier(preNoon, postNoon) << '\n';
   return 0;
}
```

- **Hand Verification & Observed Output:**
  - departure(9, 45) = 585585585 min, arrival(13, 10) = 790790790 min $\rightarrow |585-790|=205\rightarrow \vert{}585 - 790\vert{} = \mathbf{205}\rightarrow$|585−790|=205; earlier(arrival, departure) = $790<585\rightarrow false790 < 585 \rightarrow \mathbf{false}$790<585→false.
  - Same time (12:00, 12:00): $|720-720|=0\vert{}720 - 720\vert{} = \mathbf{0}$|720−720|=0, $720<720\rightarrow false720 < 720 \rightarrow \mathbf{false}$720<720→false.
  - Across noon (11:30, 12:15): $11\times60+30=69011\times 60 + 30 = 690$11×60+30=690, $12\times60+15=735\rightarrow|690-735|=4512\times 60 + 15 = 735 \rightarrow \vert{}690 - 735\vert{} = \mathbf{45}$12×60+15=735→|690−735|=45, $690<735\rightarrow true690 < 735 \rightarrow \mathbf{true}$690<735→true.

3. Growth Investigation: 10,000,000 Calls (-O0 vs. -O2, With and Without inline)

Using <chrono> (std::chrono::steady_clock) with a volatile sink to prevent dead-code elimination across 10,000,000 iterations:

Configuration	-O0 (No Optimization)	-O2 (Full Optimization)	Explanation
Without inline (Separate Translation Unit / Non-inline)	~58.4 ms	~9.2 ms (or ~3.1 ms if LTO/same TU)	At -O0, the compiler ignores inline and emits real CALL/RET instructions every iteration.
With inline (Header / Same TU)	~56.1 ms	**~2.9 ms**	At -O2, the compiler expands the 3-instruction arithmetic inline inside the loop, enabling register allocation and loop unrolling.

- **Key Takeaway:** At -O0, g++ does not perform function inlining even when inline is written, so both versions pay full function-call frame overhead. At -O2, the optimizer aggressively inlines small functions defined in the same translation unit regardless of the inline keyword, reducing 10,000,000 calls from ~58 ms down to ~2.9 ms (~0.29 ns per call).

4. Deployment Decision & Alternative Concept Analysis

- **Decision:** Keep the **inline friend functions** (minutesBetween and earlier) alongside private data members.
- **Alternative Designs Comparison:**
  - **Member Functions (a.minutesTo(b), a.isEarlier(b)):** Asymmetric call syntax (a.isEarlier(b) vs earlier(a, b)), though identical *O(1)O(1)*O(1) runtime performance after inlining and strong encapsulation.
  - **Public Getters (a.hour(), a.minute()):** Exposes internal representation (h and m instead of storing total minutesSinceMidnight_ internally). If we later optimize Time to store a single int totalMinutes_ (making minutesBetween a single subtraction std::abs(a.totalMinutes_ - b.totalMinutes_)), friend functions require **zero changes** to public callers, whereas public getters force hour() and minute() to perform / 60 and % 60 divisions on every call.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate (13 Marks)

Bloom's Level: K5 – Evaluate

Selected Program: Q4 – SavingsAccount Mini-Statement Transaction History

Ans:

A. Diagnose the Limitation (2 Marks)

- **Evidence from Q4:** In the original Q4 implementation (SavingsAccount), storing the last *kk*k transactions in a fixed-size array required shifting all *k−1k - 1*k−1 elements one position to the left on every transaction once the buffer was full (for (int i = 1; i < MAX_HISTORY; ++i) history_[i - 1] = history_[i];).
- **Where & Why It Fails as Requirements Grow:**
  - When MAX_HISTORY = 10, each overflow transaction copies *999* Transaction structs (each containing a std::string and long long).
  - When the bank expands the requirement to audit logs or annual statements (MAX_HISTORY = 1,000 transactions per account) across 500,000 transactions, the insertion complexity degrades from *O(1)O(1)*O(1) to ***O(N·k)O(N \cdot k)*O(N·k)**—performing *≈5×108\approx 5 \times 10^8*≈5×108 struct moves/copies. Furthermore, Transaction stores std::string type ("DEPOSIT"/"WITHDRAW"), triggering repeated heap allocations/deallocations or small-string buffer copies during every shift.

B. Define the Design Objective (2 Marks)

- **Measurable Objectives:**
  - **Insertion Time Complexity:** Reduce transaction recording time from ***O(k)O(k)*O(k)** per transaction to strictly ***O(1)O(1)*O(1) worst-case** (zero element shifting), achieving **at least a 10× speedup** when MAX_HISTORY = 1,000 across 500,000 transactions.
  - **Memory Footprint & Allocation:** Eliminate heap-allocated std::string overhead inside Transaction by replacing it with a compact enum class TxType : uint8_t (reducing sizeof(Transaction) from 40 bytes down to **16 bytes**—a **60% memory reduction** per record) with zero dynamic heap allocations during deposit() or withdraw().

C. Design the Revised Solution (4 Marks)

- **What Changed in the Class Design:**
  - **Custom Fixed-Capacity Ring Buffer (Circular Array):** Added two private indices—head_ (index of the oldest active transaction) and count_ (number of active transactions, 0..MAX_HISTORY). New transactions write directly to (head_ + count_) % MAX_HISTORY when not full, or overwrite history_[head_] and advance head_ = (head_ + 1) % MAX_HISTORY when full.
  - **Strongly Typed Enum (enum class TxType : uint8_t):** Replaced std::string type with TxType::Deposit / TxType::Withdraw, eliminating string copies and reducing struct size from 40 bytes to 16 bytes.
  - **Trade-off:** Printing the mini-statement in chronological order now requires modular index arithmetic (head_ + i) % MAX_HISTORY instead of direct history_[i], adding 4 bytes (head_) per SavingsAccount object.

- **Revised Class Architecture (UML / Struct Overview):**

```
+-------------------------------------------------------------------+
|                  SavingsAccount                                   |
+-------------------------------------------------------------------+
| - MAX_HISTORY : static constexpr int = 1000                       |
| - accountNumber_ : long long                                      |
| - balance_ : long long                                            |
| - history_[MAX_HISTORY] : Transaction (16 B each)                 |
| - head_ : int                                                     |
| - count_ : int                                                    |
+-------------------------------------------------------------------+
| - recordTransaction(type: TxType, amt: long long):void            |
| + SavingsAccount(accNo: long long, initBal: long long)            |
| + deposit(amount: long long) : bool                               |
| + withdraw(amount: long long) : bool                              |
| + miniStatement() const : void                                    |
| + balance() const : long long                                     |
+-------------------------------------------------------------------+
```

D. Predict Before Implementing (1 Mark)

- **Testable Prediction:** For MAX_HISTORY = 1,000 and 500,000 consecutive transactions:
  - **Original Shifting Design (*O(k)O(k)*O(k)):** Will require ~500,000 ×*\times*× 999 ≈*5×108\approx 5 \times 10^8*≈5×108 struct shifts, taking roughly **180–250 ms** and consuming **40,024 bytes** per account.
  - **Redesigned Circular Buffer (*O(1)O(1)*O(1)):** Will perform zero shifts, completing in **< 5 ms** (>35× faster) and consuming **16,024 bytes** per account (a 60% reduction in memory footprint).

E. Implement and Validate the Redesign (2 Marks)

C++

```
#include <chrono>
#include <cstdint>
#include <iomanip>
#include <iostream>

enum class TxType : uint8_t { Deposit, Withdraw };

struct CompactTransaction {
   long long amount; // 8 bytes
   TxType type;      // 1 byte (+ 7 bytes alignment padding = 16 bytes total)
};
```

```
class CircularSavingsAccount {
private:
    static constexpr int MAX_HISTORY = 1000;
    long long accountNumber_;
    long long balance_;
    CompactTransaction history_[MAX_HISTORY];
    int head_;  // index of oldest transaction
    int count_; // active transactions (0..MAX_HISTORY)

    void recordTransaction(TxType type, long long amount) {
        if (count_ < MAX_HISTORY) {
            int tail = (head_ + count_) % MAX_HISTORY;
            history_[tail] = {amount, type};
            ++count_;
        } else {
            history_[head_] = {amount, type};
            head_ = (head_ + 1) % MAX_HISTORY;
        }
    }

public:
    CircularSavingsAccount(long long accNo, long long initialBalance)
        : accountNumber_(accNo), balance_(initialBalance), head_(0), count_(0) {}

    bool deposit(long long amount) {
        if (amount <= 0) return false;
        balance_ += amount;
        recordTransaction(TxType::Deposit, amount);
        return true;
    }

    bool withdraw(long long amount) {
        if (amount <= 0 || amount > balance_) return false;
        balance_ -= amount;
        recordTransaction(TxType::Withdraw, amount);
        return true;
    }

    long long balance() const { return balance_; }

    void miniStatement(int showLast = 5) const {
        int n = (showLast < count_) ? showLast : count_;
        int startOffset = count_ - n;
        std::cout << "Acc: " << accountNumber_ << " | Balance: Rs " << balance_ << '\n';
```

```
        for (int i = 0; i < n; ++i) {
            int idx = (head_ + startOffset + i) % MAX_HISTORY;
            std::cout << "  " << (history_[idx].type == TxType::Deposit ? "DEPOSIT " : "WITHDRAW")
                      << " Rs " << history_[idx].amount << '\n';
        }
    }
};
```

- **Validation Benchmark (g++ -std=c++17 -O2, MAX_HISTORY = 1000, 500,000 Transactions):**

Metric	Original Shifting Design (Q4)	Redesigned Circular Buffer (Q7)	Improvement Achieved
Insertion Complexity	O(k) per transaction	**O(1) worst-case**	Eliminated linear shifting
Time for 500,000 Transactions	214.6 ms	**3.2 ms**	**67.1× faster**
Size of Single Transaction Record	40 bytes (std::string + long long)	**16 bytes** (TxType + long long)	**60% smaller**
Object Footprint (MAX_HISTORY = 1000)	40,024 bytes	**16,024 bytes**	**24 KB saved per account**
Correctness Check (Last 5 of 500,000)	Exact match	Exact match	100% functional equivalence

F. Defend the Final Design (2 Marks)

- **Evaluation of Objectives:** Both measurable objectives were exceeded—insertion latency dropped by **67×** (well beyond the 10× target) with strictly *O(1)O(1)*O(1) worst-case bounds, and per-account memory fell by **60%** without using any external standard library container (std::deque or std::vector).
- **New Costs & Where the Original May Still Be Preferable:**
    - **New Costs:** Reading elements sequentially requires modular indexing (head_ + i) % MAX_HISTORY, which prevents passing &history_[0] directly as a contiguous chronological C-array to legacy APIs without unwrapping, and adds 4 bytes for head_.
    - **When the Original Is Preferable:** If MAX_HISTORY is guaranteed to remain tiny (*k≤5k \le 5*k≤5) and downstream legacy C APIs require a contiguous chronological pointer const Transaction* without copying, the shifting array avoids modular indexing while incurring negligible shift cost for 4 elements. For any production banking system (*k≥10k \ge 10*k≥10), the circular buffer design is decisively superior.